

Faculty of Science

Department of Computer Science

University Publication Management System

Authors

Lebogang Keiponye 201704706

Mmoloki Morebodi 201701407

Thabo Motlhagodi 201704563

Popagano Batendi 201006068

Onalenna Sarefo 202606780

Zenzo Maeza 202003829

Table of Contents

Introduction 3

Problem Solution 3

Project Aim and Objectives 3

Aim 3

Objectives 3

Requirements Specifications 4

Functional Requirements 4

Non-functional Requirements 4

Business Rules and Constraints 5

Pseudo Requirements 5

Use cases 6

Initial Domain Model 12

Class Diagram 12

Requirements Prioritization 12

Design Goals 13

Subsystem Decomposition Diagram 18

System Architecture 21

overview 21

Presentation tier 21

Application tier 22

Data tier 22

Class/ Domain Model 24

Design Specification 27

Working Core functionalities 28

Test Cases 29

Requirements Traceability 32

Additional Functionalities Working 38

INDIVIDUAL CONTRIBUTIONS 40

Introduction

The purpose of this document is to specify the initial requirements and object-oriented analysis for a web-based University Publication Management System. Problem Statement

Currently the University of Botswana has no central system for storing and managing research articles, PhD articles, MPhil/MSc dissertations as the university lecturers publish them in journals, conferences and workshops. As a result, research outputs are difficult to locate, there's no search mechanism and visibility to students, UB lecturers and external users is limited.

Problem Solution

The proposed web-based University Publication Management System will address these challenges by providing a single platform for capturing publication metadata and associated documents, organizing publications by authors and subject areas, supporting search and retrieval, and providing controlled publication verification and reporting.

Project Aim and Objectives

Aim

To analyze, design, implement, and test a secure web-based publication system that centralizes the management, discovery and controlled access of university research publications, thesis, and dissertations.

Objectives

The system should:

1. Store university publications online.
2. Support uploading publications.
3. Support publication searching, filtering, viewing, and retrieval.
4. Manage authors and publication details.
5. Support approval or verification of publications.
6. Generate the publication reports.
7. Provide user registration, authentication, and role-based access control.
8. Capture publication metadata.

Requirements Specifications

The requirements define the functional and non-functional capabilities that the University Publication Management System must provide.

Functional Requirements

These requirements define what the system should.

- The system shall allow users to register an account where registration is permitted by the university policy.
- The system shall allow users to login using valid credentials.
- The system shall allow users to search for publications.
- The system shall support advanced search using supported criteria such as author, title, subject /area.
- The system shall enforce role-based access control to the user's assigned role.
- The system shall allow authorized users to create publication records.
- The system shall allow users to view and update their profiles subject to their permissions.
- The system shall allow authorized users to download or view an accessible publication document.
- The system shall allow researchers to upload publication documents.
- The system shall allow users to edit publication records subject to their permissions.
- The system shall allow users to submit a publication for verification.
- The system shall allow reviewers or approvers to view submitted articles awaiting verification.
- The system shall allow authorized verifiers to approve and reject submitted publications.
- The system shall generate publication reports based on the requested criteria.
- The system shall allow users to track the status of publications they have submitted.
- The system shall store publication status, actions such as submission, approval, rejection, and deletion of publications for accountability.
- The system shall notify the appropriate user when the publication submission is approved or rejections through notifications.

Non-functional Requirements

These requirements define the qulaity attributes, performance characteristics and constraints that the system shall satisfy.

- Search results shall be displayed within 3 seconds for 95% of queries.

- Only authenticated users shall upload publications.
- The system shall provide at least 99% availability, excluding planned maintenance, and it should be communicated to users in advance.
- Users shall be able to complete the publication process in not more than 5 user actions/steps, excluding authentication.
- The system shall maintain the integrity of stored publication and user data and shall prevent accidental loss during normal system operation.
- Passwords shall be stored using secure password hashing mechanisms.
- The interface shall provide clear navigation and understandable validation or error messages.
- The design shall scale from its initial baseline of 500 users, 1000 publication records, and 1000 stored documents to 5000 users and 10000 records and documents within five years, without requiring fundamental changes to the system architecture.
- The system shall restrict access to private or non-public publications to authorized users.
- The system should accurately store and restore publication data without accidental loss of data.

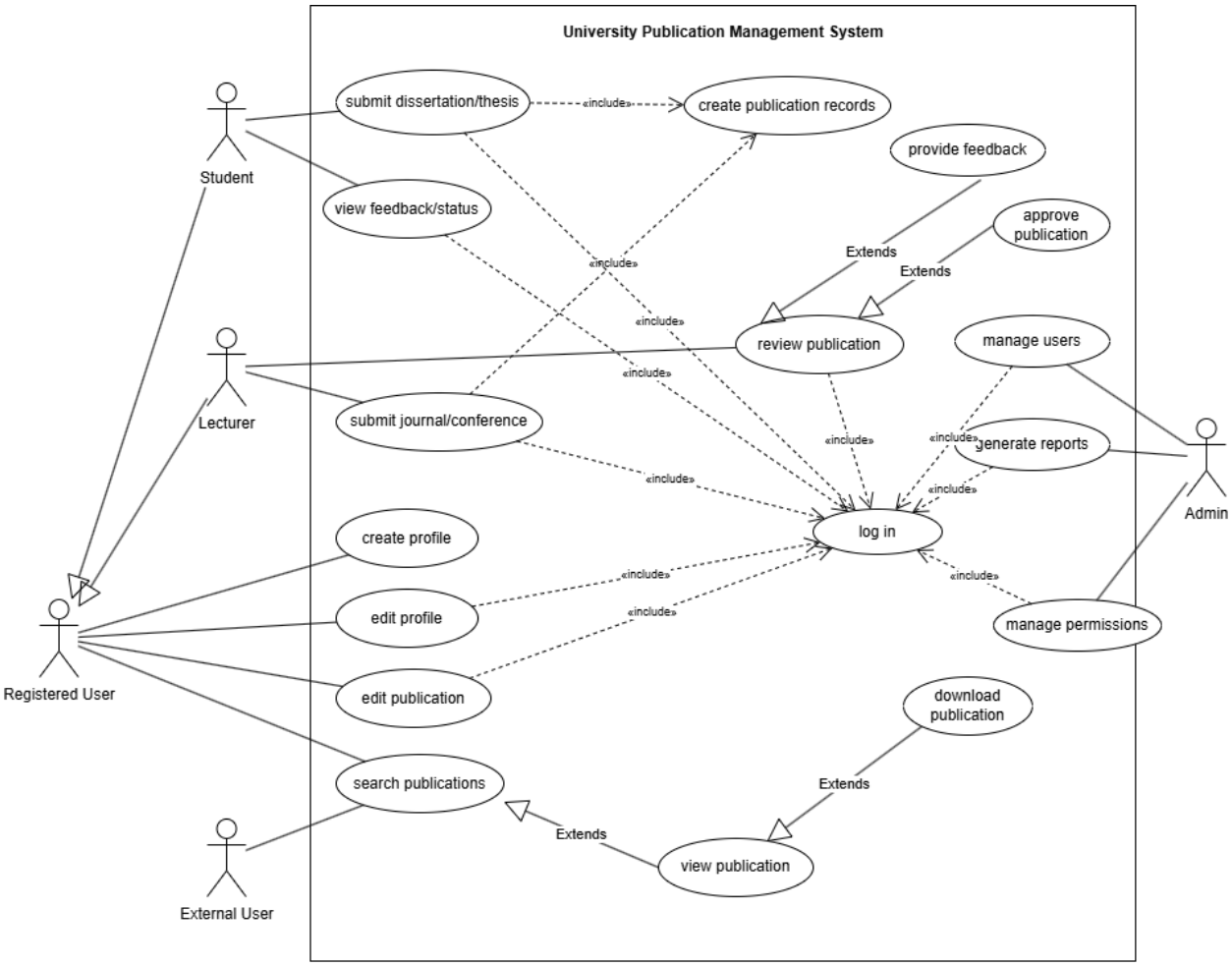
Business Rules and Constraints

- Only authenticated and authorised users shall be permitted to perform restricted management functions.
- External users shall only be permitted to access the publications designated publicly accessible.
- A publication shall pass the defined verification workflow before it can be approved for publication.
- Users shall only be permitted to edit or delete the publication records for which their role grants permission.
- Only authorized verifiers shall be permitted to approve or reject submitted publications.

Pseudo Requirements

- The system shall be developed using version control throughout the development (e.g., Git).

Use cases


University Publication Management System
Student
Lecturer
Registered User
External User
Admin
submit dissertation/thesis
create publication records
view feedback/status
provide feedback
approve publication
review publication
manage users
submit journal/conference
generate reports
log in
create profile
edit profile
edit publication
search publications
download publication
view publication
manage permissions
«include»
Extends

USE CASE DESCRIPTIONS

Use Case: Login

Participating Actor(s)	Lecturer, Student, Reviewer, Admin
Precondition	The user has an existing account
Main Flow	1. The user enters username and password 2. The system validates the credentials 3. The system identifies the user's role 4. The system grants access to the appropriate functionality
Postcondition	The user is successfully authenticated and logged into the system
Exceptions	Invalid username/password
Special Requirements	Authentication must be secure

Use Case: Create Publication Record

Participating Actor(s)	Lecturer, Student
Precondition	Authorized user logged in
Main Flow	1. The user selects "create publication record" 2. The system displays publication form 3. The actor enters the publication title 4. The actor enters the publication details 5. The actor enters the author 6. The actor specifies the subject/area 7. The actor clicks "submit" 8. The system validates the entered information 9. The system stores the publication record 10. The system executes <<include>> Upload Publication Record to attach the associated file with the newly created record.
Postcondition	New publication record created and submitted

Exceptions	Required information is missing Duplicate Record

Use case: Upload Publication Record

Participating Actor(s)	Lecturer, Student
Precondition	Authorized user logged in
Main Flow	11. The user selects “upload file” 12. The system displays upload interface 13. The user uploads publication 14. The user clicks “submit” 15. The system validates the file 16. The system stores the file
Postcondition	New file for publication uploaded
Exceptions	Unsupported filetype File is too large

Use case: Submit Publication

Participating Actor(s)	Lecturer
Precondition	A new publication record has been created and stored in the system with the status "Pending review"
Main Flow	1. The use selects “Submit Publication” 2. The system displays Submit Publication interface 3. The system validated all the mandatory fields. 4. The system changes the status to “Submitted” 5. The system notifies the actor of successful submission
Postcondition	A publication is submitted

Use Case: Generate Reports

Participating Actor(s)	Admin
Precondition	Admin logged in
Main flow	1. The admin selects “Generate Report” 2. System compiles publication data 3. The system displays or exports the report
Postconditions	A publication report is produced for the admin

Use case: Review Publication

Participating Actor(s)	Reviewer
Preconditions	Reviewer logged in and a publication submitted for review
Main flow	1. The reviewer selects “manage publications” 2. The reviewer opens a submitted publication and examines its details 3. Reviewer approves or rejects the publication 4. The system updates the publication status 5. The system notifies the author of the decision
Postconditions	The publication status is updated to "approved" or "rejected", and author is notified
Exceptions	1. The reviewer is not authorized to review the publication; access is denied.
Special Requirements	1. Only the authorised reviewer may approve or reject the assigned publication. 2. The system must retain the review decision and publication status for accountability. 3. The author must be notified of the review decision as already specified in the main flow.

Alternative flow	Reviewer rejects the publication; the author may edit and resubmit it

Use Case: Track Publication

Participating Actor(s)	Lecturer, Student
Precondition	The actor is logged in and has publication submitted to the system
Main flow	1. The actor selects Track Application Status 2. The system retrieves the publication 3. The system displays the status and relevant status information
Postconditions	The actor knows the current status of the publication

Use case: View/Download Publication

Participating Actor(s)	Lecturer, Student, Reviewer, External User
Precondition	Publication approved
Main Flow	1. The user selects publication from search results 2. The system displays the publication allows the document to be downloaded
Postcondition	The publication is viewed or downloaded by the user

Use case: Search Publications

Participating Actor(s)	Lecturer, Student, External User
Precondition	The publication repository is available
Main flow	4. The actor selects Search Publications 5. The actor enters search criteria 6. The system searches the publication repository 7. The system identifies matching publications

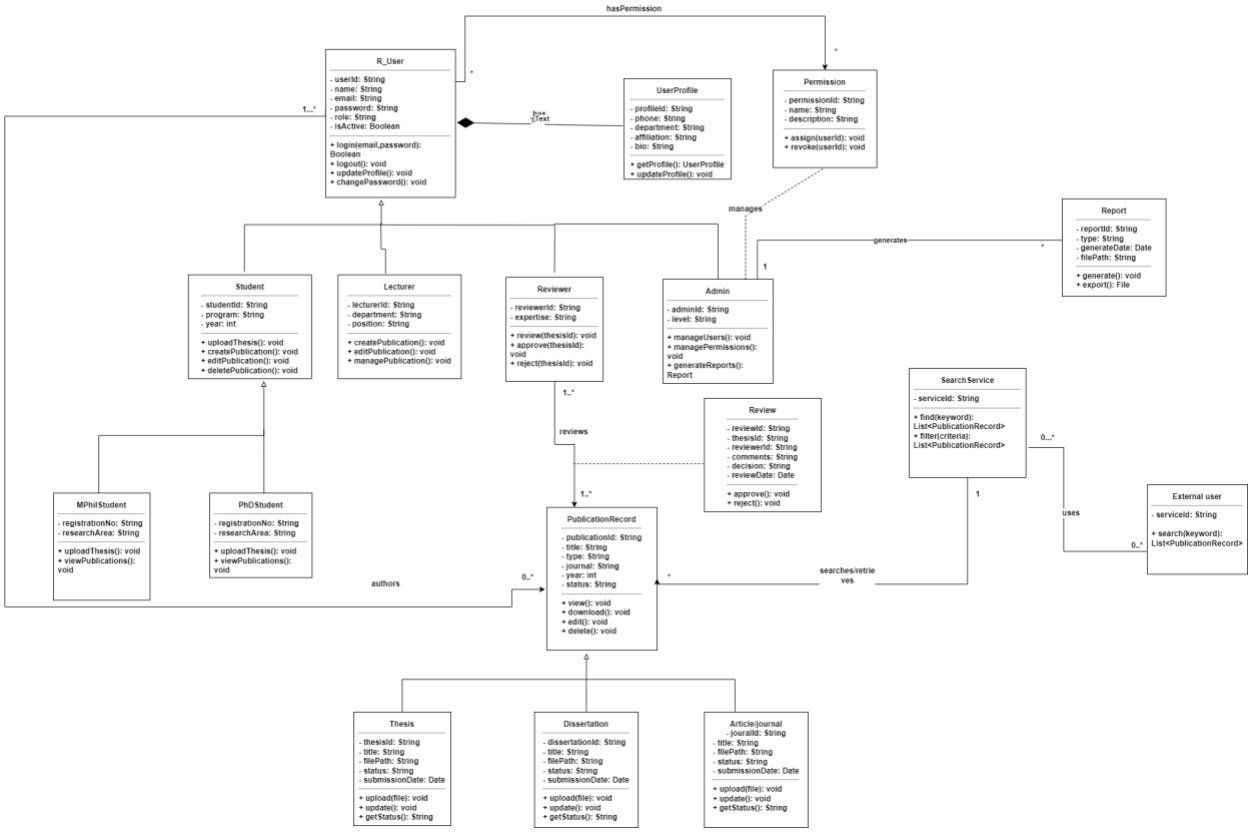
	8. The system displays the search results 9. The actor selects a publication if required
Postconditions	The system displays publications matching the search criteria

User case: Generate Report

Participating Actor(s)	Admin
Precondition	Admin logged in
Main flow	10. The admin selects “Generate Report” 11. System compiles publication data 12. The system displays or exports the report
Postconditions	A publication report is produced for the admin

Initial Domain Model

Class Diagram



Requirements Prioritization

is used to determine what should be implemented first.

Must have:

- The system shall authenticate users using valid credentials (login).
- The system shall allow researchers to upload research papers.
- The system shall allow users to search publications.
- The system shall allow authorized users to download or view an accessible publication document.
- The system shall enforce role-based access control.
- The system shall allow users to create a publication record.

- The system shall store publication status, actions such as submission, approval, rejection, and deletion of publications for accountability.

The system Should have:

- Support advanced search using one or more supported criteria.
- Generate publication reports based on the requested criteria.
- Allow users to view and update their profiles subject to their permissions.
- Notify the appropriate user when the submitted publication is approved or rejections through notifications.
- Allow users to track the status of publications they have submitted.

The system Could Have:

- Publication recommendations.
- Advanced research statistics dashboards.
- Export formats for reports.
- Citation generation.

The system Won't have

- Mobile Application.
- AI-based search
- Plagiariam detection

Design Goals

Design Goals for the University Publication Management System

1. **Security - Highest Priority**

Design goal: The publication system should protect user accounts, publication records, and restricted publication documents from unauthorized access or modification.

Related requirements / stakeholder concerns:

- Only authenticated users may upload publications.
- Passwords must be stored securely.
- The system must enforce role-based access control.
- Only authorized reviewers or administrators may approve, reject, or manage protected records.

Design implications:

The system should use authentication and role-based authorization before allowing protected operations. Access checks should be performed in the Application Tier rather than relying only on hiding buttons in the user interface. Uploaded documents should also be validated before storage.

Trade-off: Security vs. usability and rapid development

Stronger security introduces additional login, validation, and permission checks. These controls may increase implementation complexity and may require users to perform additional steps. However, security is prioritized because the system handles user accounts and potentially restricted university publications.

2. Maintainability

Design goal: The system should be structured so that changes to one area of functionality can be made with minimal impact on unrelated parts of the system.

Related requirements / stakeholder concerns:

- Publication types and subject areas may change over time.
- Verification and approval processes may evolve.
- User roles and permissions may require future adjustments.
- The development team needs a system that is easy to modify, understand, and test.

Design implications:

The publication should be decomposed into coherent subsystems such as verification and approval and publication management. Each subsystem should have a focused responsibility and communicate through well-defined services or subsystem interfaces.

For example, changing the approval rules should mainly affect verification and approval rather than requiring changes to a different subsystem.

Trade-off: Maintainability vs. rapid development

A modular design requires more initial planning and may take longer to implement than placing most functionality together. However, it reduces the cost and risk of future changes.

3. Usability

Design goal: The publication system should provide a simple and understandable web interface that allows its different users to perform their tasks with minimal difficulty.

Related requirements / stakeholder concerns:

- Users must be able to register and log in.
- Researchers must be able to upload publications.
- Users must be able to search, filter, view, and download publications.
- Reviewers must be able to view pending submissions and make decisions.
- The upload process should require no more than five user actions or steps, excluding authentication.
- Navigation and error messages should be clear.

Design implications:

The system should provide role-appropriate interfaces and avoid exposing unnecessary functions to users. For example reviewer dashboard should include:

- Pending publications
- Review
- Approve / Reject

Trade-off: Usability vs. functionality/security

Adding many features to a single screen may make the system more powerful but harder to understand. Similarly, additional security checks may increase the number of interactions required from users. The proposed solution should therefore balance ease of use with necessary security and functionality.

4. Reliability and Data Integrity

Design goal: Publication system should preserve publication and user information accurately and remain dependable during normal system operation.

Related requirements / stakeholder concerns:

- The system must maintain the integrity of stored information.
- Accidental loss of publication information should be prevented.
- Regular backups should support recovery.
- Publication status and verification of decisions must remain consistent.
- Significant actions should be recorded for accountability.
- The University needs confidence that uploaded research information will not be lost or incorrectly modified.

Design implications:

The proposed system should use persistent database storage for metadata and controlled file storage for publication documents. The system should also preserve verification history rather than overwriting previous decisions. For example:

Publication:

- Verification Record 1 → Rejected
- Verification Record 2 → Approved

Trade-off: Reliability vs. cost and performance

Backups, validation, audit logging, and recovery mechanisms require additional storage and processing resources. These mechanisms may increase infrastructure cost and processing overhead, but they are necessary to protect university's research records.

5. Performance and Scalability

Design goal: The system should respond efficiently to user requests and continue to operate effectively as the number of users, publications, and uploaded documents increases.

Related requirements / stakeholder concerns:

- Search results should be returned within approximately three seconds under normal operating conditions.
- The system should support growth in users, publications, and documents.
- Users expect publication searches and retrieval to be responsive.
- The University may continuously add new publications over time.

Design implications:

The architecture should separate presentation, application processing, and persistent data management. Search functions should query appropriate metadata rather than unnecessarily loading complete publication documents. Database indexes can later be applied to commonly searched fields such as:

Trade-off: Performance vs. cost and maintainability

Higher performance can require more powerful infrastructure, caching, indexing, or additional optimization. These increase cost and implementation complexity. The design should therefore meet the required response time without introducing unnecessary optimization.

Why this priority order?

Security is given the highest priority because the system manages user accounts, role-based functionality, and potentially restricted research documents. Maintainability is second because publication of workflows, verification rules, and university requirements may change over time. Usability is third because the system will be used by different categories of users with different levels of technical experience. Reliability and data integrity are prioritized to ensure that publication records and verification history are not lost or corrupted. Performance and scalability are also important because publication searches must remain responsive, and the amount of stored research content is expected to grow.

Subsystem Decomposition Diagram

UniPub System Decomposition

Below, each layer is decomposed into its subsystems, followed by the services each one provides.

1. **Decomposition overview**

Layer	Subsystem / Component	Role
Presentation	UniPub System (client interface)	Entry point for users, communicating over HTTPS/JSON
Application	Backend API Layer	Single gateway that routes requests to the business subsystems
Application	User Management	Accounts, authentication, access control
Application	Publication Management	Creation and maintenance of publication records
Application	Verification & Review	Review and approval of submitted publications
Application	Search & Discovery	Browsing, filtering, retrieval and download
Application	Reporting Subsystem	Reports and analytics
Data	Database	Structured records (users, publications, review data)
Data	Document Store	Full-text publication files

2. **Subsystem services**

Presentation Layer - UniPub System (client interface)

- Provides the user-facing interface for all roles (authors, reviewers, administrators, readers).
- Sends requests to and receives responses from the Backend API over HTTPS using JSON.
- Contains no business logic; it only presents data and collects input.

Application Layer - Backend API Layer

- Single entry point for every request from the presentation layer.
- Routes each request to the correct subsystem and returns the result as JSON.

- Gives the subsystems a common interface, so the interface and business logic stay decoupled.

User Management

- Registration: creates new user accounts.
- Authentication: verifies credentials and manages login sessions.
- Permissions: enforces role-based access (e.g. who may submit, review, or administer).
- User profile: stores and updates user details.
- Accesses the Database through the ORM.

Publication Management

- Handles the publication record lifecycle: submit, edit, update, and delete.
- Links each publication to its author (via User Management) and passes submissions on to Verification & Review.
- Stores metadata in the Database (and indexes it) and stores the document file in the Document Store (store/retrieve).
- Serves publication data to other subsystems through getPublication().

Verification & Review

- Review: lets reviewers examine submitted publications.
- Verification: records the decision (approve, reject, or request changes) and updates the publication's status.
- Retrieves publication data through getPublication() and returns the outcome to Publication Management.

Search & Discovery

- viewPublications: lists and displays verified publications.
- filterPublications: narrows results by criteria such as author, title, year, or keyword.
- download: retrieves the full document from the Document Store.
- Uses the search index in the Database so queries stay fast.

Reporting Subsystem

- Report: generates summaries such as publications per author, department, or period, and review status.
- Analytics dashboard presents usage and publication statistics.

- Reads from User Management and Publication Management data; it does not modify records.

Data Layer -Database

- Persists structured data: user accounts, publication metadata, review status, and the search index.
- Accessed by the application subsystems via the ORM (object-relational mapping).

Document Store

- Stores the full-text publication files (e.g. PDFs).
- Supports store and retrieve operations, used by Publication Management (upload) and Search & Discovery (download).

3. **Key interactions**
    - Submission: UniPub System → Backend API → Publication Management → Database (metadata) and Document Store (file).
    - Review: Publication Management → Verification & Review (via getPublication()) → status returned to Publication Management.
    - Discovery: UniPub System → Backend API → Search & Discovery → search index (Database) → Document Store for downloads.
    - Reporting: Reporting Subsystem aggregates data from User Management and Publication Management.

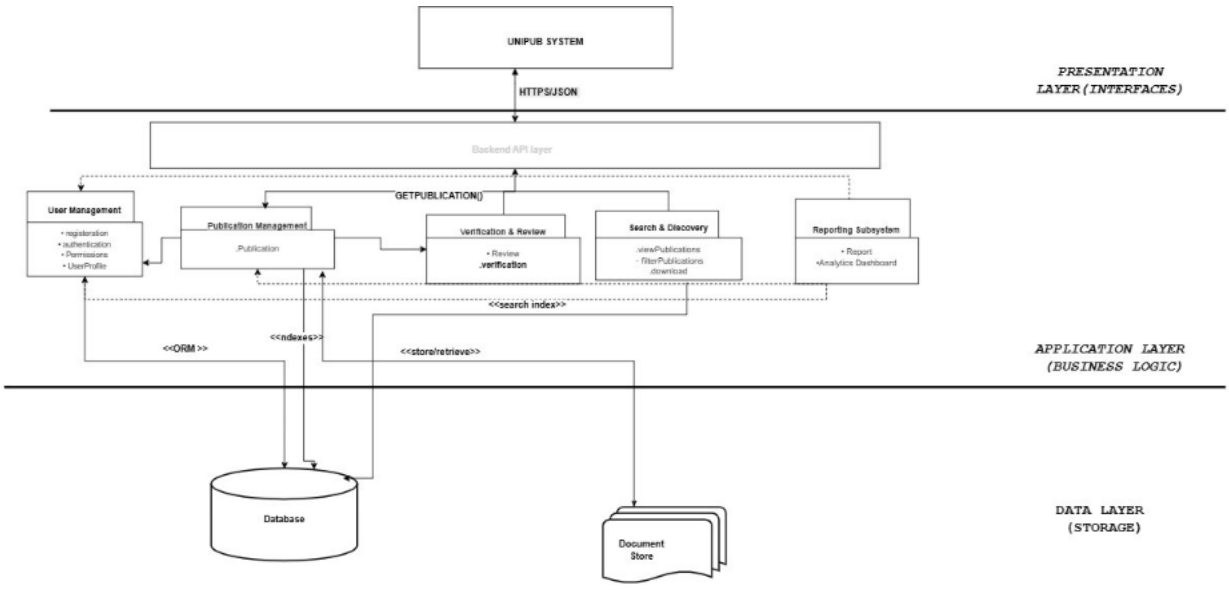


Figure 1: The diagram shows a three-layer architecture: Presentation, Application (business logic) and Data.

System Architecture

overview

The University Publication Management System will adopt a 3-Tier architecture.

The three tiers are:

1. Presentation tier
2. Application tier
3. Data tier

Within the Application tier, the Django framework will provide separation of responsibilities using Model, Template, and View (MTV) architectural pattern, which is conceptually related to MVC.

The separation of the web application into tiers allows user interface, application processing, and data storage responsibilities to be managed independently. This improves maintainability, modularity, and separation of concerns.

Presentation tier

The presentation tier represents the front-end layer of the system. It provides the graphical user interface accessed through a browser to display content to UB lecturers, students, external users, administrators, and reviewers.

It captures the user's inputs and displays query results. Examples of interfaces provided through the presentation tiers include:

- User registration and login page
- User dashboards
- Publication search page
- Publication upload form
- Publication reports
- Publication search

The presentation tier does not directly implement business rules or communicate directly with the database. Instead, user requests are forwarded to the Application tier for processing.

Application tier

It consists of application and business logic of the system. It processes requests received from the presentation tier, applies business rules, validates user actions, enforces access permissions, and coordinates communication with the Data tier.

For example, when a researcher uploads a publication, the application tier will:

- Verify that the researcher is authenticated and authorized.
- validate the publication information and document.
- Create the publication record,
- Initiate the appropriate publication verification workflow.

The system will be implemented using Django's Model, Template, and View pattern. The major Django components are:

Model (models.py) - they represent and manage application data such as users, publications, authors, subject areas, publication types and verification records.

View (views.py) - receives HTTP requests, coordinates application logic, accesses models where necessary, and returns the appropriate response.

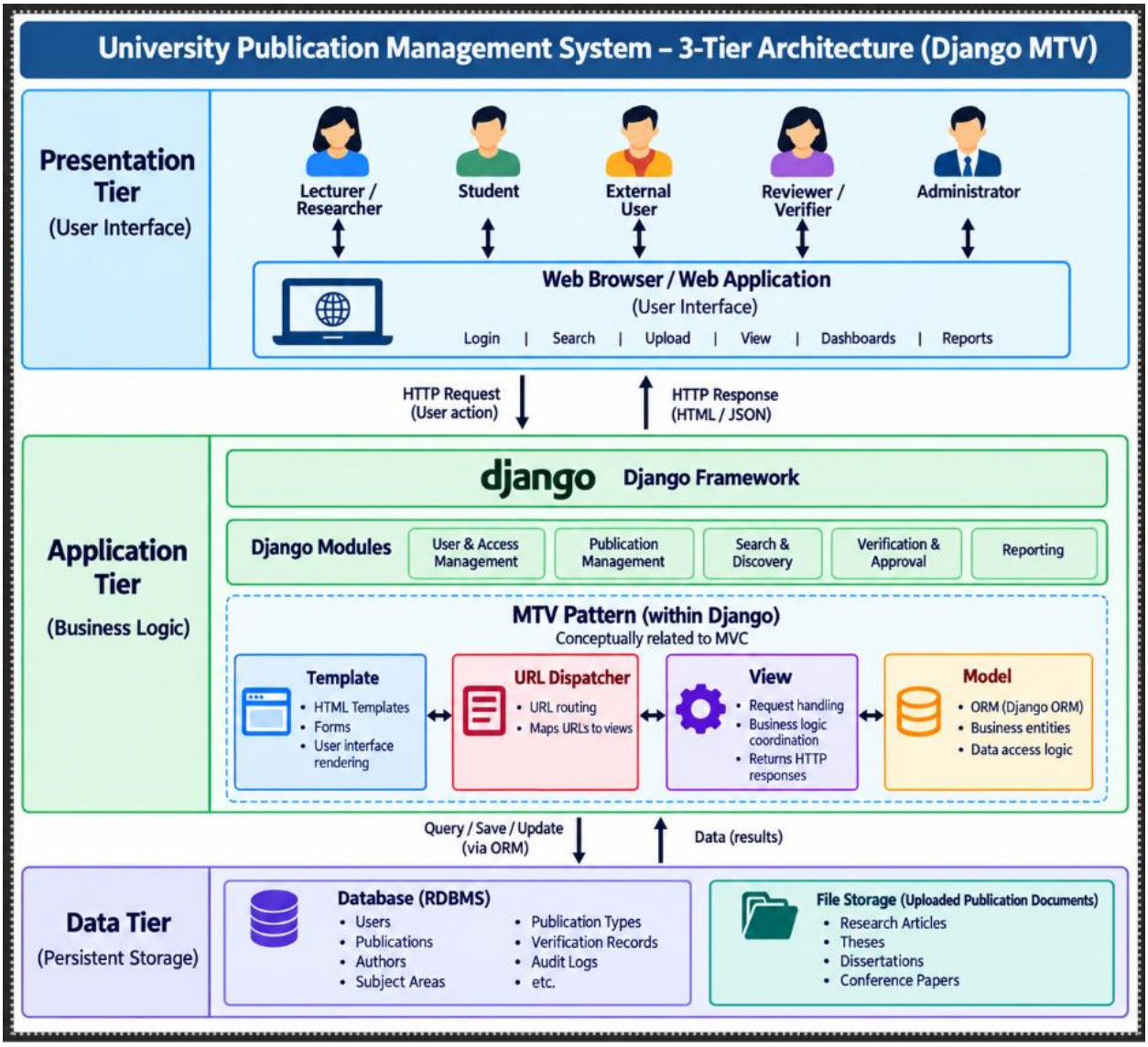
Template (templates/) - defines how information is represented to users through HTML pages. URL Dispatcher (urls.py) - maps incoming URLs to the appropriate Django view.

Data tier

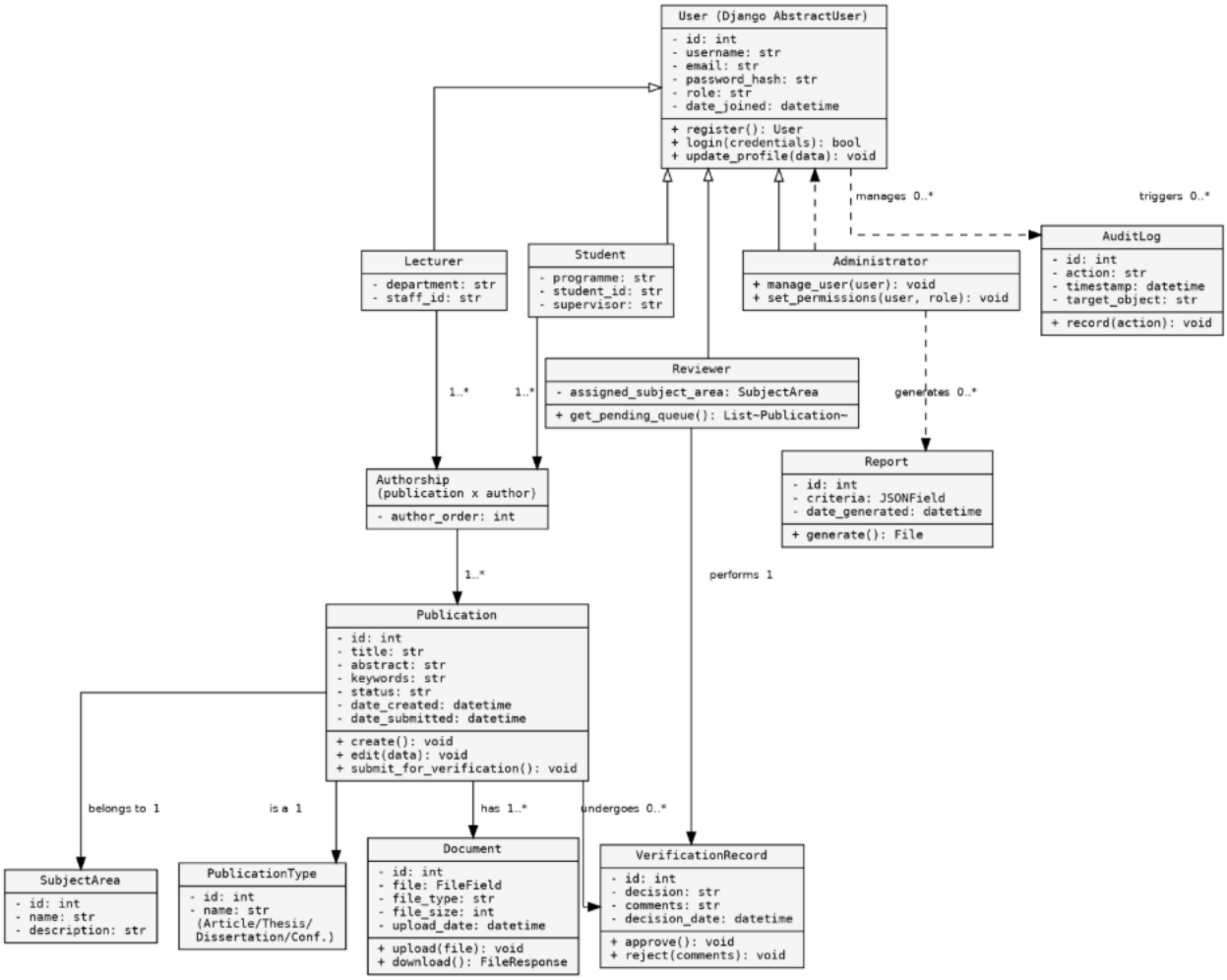
The data tier is responsible for persistent storage and retrieval of system data. The proposed system will use a relational database management system (MySQL) to store structured information such as:

- Users
- Roles and permissions
- Verification records
- Publication status and others

The data tier will also include file storage for uploaded publication documents such as: research articles, thesis, dissertations, and conference papers.


University Publication Management System – 3-Tier Architecture (Django MTV)
Presentation Tier
(User Interface)
Lecturer / Researcher
Student
External User
Reviewer / Verifier
Administrator
Web Browser / Web Application
(User Interface)
Login | Search | Upload | View | Dashboards | Reports
HTTP Request (User action)
HTTP Response (HTML / JSON)
Application Tier
(Business Logic)
django Django Framework
Django Modules
User & Access Management
Publication Management
Search & Discovery
Verification & Approval
Reporting
MTV Pattern (within Django)
Conceptually related to MVC
Template
HTML Templates
Forms
User interface rendering
URL Dispatcher
URL routing
Maps URLs to views
View
Request handling
Business logic coordination
Returns HTTP responses
Model
ORM (Django ORM)
Business entities
Data access logic
Query / Save / Update (via ORM)
Data (results)
Data Tier
(Persistent Storage)
Database (RDBMS)
Users
Publications
Authors
Subject Areas
Publication Types
Verification Records
Audit Logs
etc.
File Storage (Uploaded Publication Documents)
Research Articles
Theses
Dissertations
Conference Papers

Class/ Domain Model



Core Submission Workflow

1. Lecturer/Student calls Publication.submit_for_verification() status set to pending
2. VerficationRecord is created and queued for the Reveiwer
3. Reviewer calls approve() or reject() Publication.status updated; AuditLog entry is recorded; the author is notified

Class responsibilities

Class	Responsibility
User	Base for authentication and profile management
Lecturer	One of the authoring roles can create publication records
Student	One of the authoring roles
Reviewer	Examines publications submitted within subject area for verification decision
Administrator	Responsible for user/account management and generating reports
SubjectArea	Classify publications by discipline used to route publications to a matching Reviewer
PublicationType	Distinguishes between article, thesis, dissertation, conference paper
Publication	Holds publications metadata and lifecycle sstatus coordinates submission workflow
Document	Uploaded file tied to a publication
VerificationRecord	Capture outcome of a review (e.g. reject or approve)
ReportService	On demand summary generated by Administrator from publication data
Authorship	Resolves many-to-many relationship between authors and publication while reserving order
AuditLog	Records significant actions for accountability

Data Dictionary (key attributes)

Attribute	Type	Meaning
User.role	str	One of: lecturer, student, reviewer, administrator
Publication.status	str	One of: draft, pending_verification, approved, rejected controls available actions and what's publicly visible
VerificationRecord.decision	str	One of: approved, rejected set once per record;

		rejection may lead to new record resubmission
Authorship.author_order	int	1 position in the author list; used for citation formatting
Document.file_type	Str	Restricted to accepted formats enforced at upload
AuditLog.target_object	str	Generic reference so log table can reference any entity type

Relationship Rationale

Authorship is an explicit class rather than a plain many-to-many field because author_order needs to be stored somewhere a bare M:N relation can't carry that extra attribute.

SubjectArea and PublicationType are separate entities, not hard-coded text fields, so new areas/types can be added by an administrator without a schema change, and so Reviewer can be matched to publications by subject area.

VerificationRecord is 0..* against Publication (not 1:1) because a publication can be rejected and resubmitted, producing a new record each time this preserves review history rather than overwriting it.

AuditLog isn't tied to a foreign key on a single table because it needs to log actions across multiple entity types uniformly.

Traceability to requirements

Requirements	Realized by
System shall authenticate users using valid credentials	User.login(), authentication mechanism
System shall allow researchers to upload research papers	Publication, Document.upload(), Document
System shall allow users to search publications	Publication, SubjectArea, Search functionality
System shall allow authorized users to download or view publications	Document.download(), Access Control
System shall enforce role-based access control	User.role, Administrator.set_permissions()
System shall allow users to submit publications for verification	Publication.submit_for_verification, VerificationRecord
System shall allow reviewers to view publications awaiting verification	Reviewer, Publication.status, VerificationRecord

System shall allow authorized verifiers to approve or reject submitted publications	Reviewer, VerificationRecord, Publication.status
System shall generate publication reports based on requested criteria	Report
System shall allow submitters to track the status of their publications	Publication.status
System shall store publication status and significant actions including submission, approval, rejection and deletion	Publication.status, VerificationRecord, AuditLog

Design Specification

Design Overview

The design specification describes how the classes classified in the domain model support the main system functions. It defines the key object operations and interactions required for publication management, verification, access control, reporting, and auditing while following the proposed three-tier architecture.

Key Object Operations

Class	Operation	Description
User	login()	Authenticates a user using valid credentials
Lecture/Student	CreatePublication()	Creates a new publication record
Publication	Submit_for_verification	Submits a publication for verification and changes its status to pending_verification
Document	upload()	Uploads a document associated with a publication
Document	download()	Retrieves an accessible publication document
Reviewer	approve()	Approves a submitted publication
Reviewer	reject()	Rejects a submitted publication
Administrator	set_permissions()	Manages user permissions
Report	generate()	Generates publication reports based on specified criteria
AuditLog	recordAction()	Records significant system actions for accountability

Design Decisions and Rationale

Design Decision	Rationale
Three-tier architecture	Separates the presentation, application, and data responsibilities, making the system easier to maintain and manage.
MVC-related separation	Separates user interface concerns from application logic and data management.
Django framework	Supports the implementation of the web application while providing structure for handling requests, application logic, and data access.
MySQL database	Provides persistent storage for structured system data such as users, publications, verification records, and audit information.
Separate file storage	Stores uploaded publication documents separately from their metadata maintained in the database.
Verification workflow	Ensures submitted publications pass through an approval or rejection process before reaching an approved state.
Audit trail	Maintains a history of significant publication actions for accountability.

Increment 1 - Working Core functionalities

The core functionalities are based on our requirements prioritization, mainly on the Must Have requirements being:

1. User login – users enter credentials and authenticate them.
2. Role-Based Access Control – different roles access only permitted functions.
3. Create publication records – lecturer/student creates publication records.
4. Upload publication – authorized users upload a valid publication file.
5. Search publications – users search publications.
6. View publications – users open and view accessible publications.
7. Download publication – downloadable publications are accessed.

Test Cases

System Name	UNIPUB							
Document No:					**Version No**:	*001*		
Name of tester:	LEBOGANG KEIPONYE				**Name of service:**	**Registration**		
Date:					**Stage:**	Increment 1- Functional Testing		
Requirement	**Test Case ID**	**Test Case Description**	**Test Steps**	**Prerequisite**	**Expected Results**	**Actual Results**	**Pass/Fail**	**Comments**
REQ001 - The system shall authenticate users using valid credentials (login).	001	Test whether a user can create an account.	1. Open UNIPUB. 2. Click sign up. 2. Enter a registered username and password, and other details. 3. Click Verify & Register. 4. Click OK.	UNIPUB is running, and registration is enabled; user details should exist in the ASAS HR profiles.	User is verified with corresponding names from the ASAS HR database and required to login.	A message Registration completed for your relevant names are displayed and required to login.	PASS	The names come as they're from the ASAS database.

System Name	UNIPUB							
Document No:					**Version No**:	*001*		
Name of tester:	LEBOGANG KEIPONYE				**Name of service:**	**Login**		
Date:					**Stage:**	Increment 1- Functional Testing		
Requirement	**Test Case ID**	**Test Case Description**	**Test Steps**	**Prerequisite**	**Expected Results**	**Actual Results**	**Pass/Fail**	**Comments**
REQ002 - The system shall authenticate users using valid credentials (login).	001	Test whether a user can login using valid credentials.	1. Open UNIPUB. 2. Click sign in. 2. Enter a registered username and password. 3. Click Authenticate Profile. 4. Click OK.	User accounts should exist.	User is authenticated and redirected to the corresponding dashboard, based on their role.	A user logged in, and a dashboard was displayed.	PASS	
	002	Test login using invalid credentials (password).	1. Enter a registered username or non-existing credentials 2. Enter an incorrect password.	User accounts already exist.	Login is rejected, "Authentication rejected. Verify profile credentials." message is displayed.	The message is displayed, and it didn't login.	PASS	

System Name	UNIPUB							
Document No:					**Version No**:	*001*		
Name of tester:	LEBOGANG KEIPONYE				**Name of service:**	**Role-Based Access Control**		
Date:					**Stage:**	Increment 1- Functional Testing		
Requirement	**Test Case ID**	**Test Case Description**	**Test Steps**	**Prerequisite**	**Expected Results**	**Actual Results**	**Pass/Fail**	**Comments**
REQ003 - The system shall enforce role-based access control.	003	Test whether a normal user can access only functionality assigned to their role.	1. Login as a student/lecturer. 2. Observe available menu options. 3. Attempt to access the functions.	User accounts with different roles must exist.	User can access functions permitted to their role.	A lecturer's dashboard, has only it's roles,	PASS	

System Name	UNIPUB							
Document No:					**Version No**:	*001*		
Name of tester:	LEBOGANG KEIPONYE				**Name of service:**	**Create Publication Record**		
Date:					**Stage:**	Increment 1- Functional Testing		
Requirement	**Test Case ID**	**Test Case Description**	**Test Steps**	**Prerequisite**	**Expected Results**	**Actual Results**	**Pass/Fail**	**Comments**
REQ004 - The system shall allow users to create a publication record	004	Test whether an authorised user can create a publication record using valid information.	1. Click Sign in. 2. Enter publication title. 3. Click the icon next to the Publication title to add the author. 4. Enter the subject area. 5. Enter Abstract Description. 6. Select Publication Type. 7. Attach the publication file. 8. Click Submit Publication.	Authorized user is logged in.	Information is validated and a new publication is successfully stored in the system.	The publication is submitted successfully.	PASS	
	005	Test creating a publication with missing required information.	1. Follow steps 1 to 4. 2. Leave a mandatory field empty. 3. Click Submit Publication.		Publication is not saved, and a message "Please fill in the required information" is displayed.	It tells the user that some information is required.	PASS	

System Name	UNIPUB							
Document No:					**Version No**:	*001*		
Name of tester:	LEBOGANG KEIPONYE				**Name of service:**	**Upload Publication Record**		
Date:					**Stage:**	Increment 1- Functional Testing		
Requirement	**Test Case ID**	**Test Case Description**	**Test Steps**	**Prerequisite**	**Expected Results**	**Actual Results**	**Pass/Fail**	**Comments**
REQ005 - The system shall allow researchers to upload research papers.	006	Test whether an authorized researcher can upload a valid publication record.	1.Login as Lecturer/researcher. 2. Create a publication record. 3. Click Upload File. 4. Choose a supported publication document. 5. Click Upload	Authorized user is logged in, and a publication record exists.	File is validated, uploaded and associated with the relevant publication record.	The file is successfully uploaded.	PASS	
	007	Test upload an unsupported file type.	1. Follow steps 1 to 3. 2. Select unsupported file type.	Authorized user is logged in.	System rejects the file and message "upload an appropriate file type."	The message is displayed.		

System Name	UNIPUB							
Document No:					**Version No**:	*001*		
Name of tester:	LEBOGANG KEIPONYE				**Name of service:**	**Search Publication Record**		
Date:					**Stage:**	Increment 1- Functional Testing		
Requirement	**Test Case ID**	**Test Case Description**	**Test Steps**	**Prerequisite**	**Expected Results**	**Actual Results**	**Pass/Fail**	**Comments**
REQ006 - The system shall allow users to search publications.	008	Test whether a system can search for an existing publication by key words.	1. Click search 2. Enter the publication title. 3. Click Search.	Atleast one of the publications exists in the repository.	The system retrieves and displays the publication matching the search criteria by title, subject area, author, etc.	The search queries are displayed.	PASS	
	009	Test whether the system handles a search with no matching publication.	1. Click search 2. Enter the publication title. 3. Click Search.	Publication repository must be available	No publication is returned, and the system indicates that no matching records were found.	No search is displayed.	PASS	

System Name	UNIPUB							
Document No:					**Version No**:	*001*		
Name of tester:	LEBOGANG KEIPONYE				Name of service:	**View and Download Publication**		
Date:					**Stage:**	Increment 1- Functional Testing		
Requirement	**Test Case ID**	**Test Case Description**	**Test Steps**	**Prerequisite**	**Expected Results**	**Actual Results**	**Pass/Fail**	**Comments**
REQ007 - The system shall allow authorized users to download or view an accessible publication document.	010	Test whether users can view an accessible published document.	1. Search for an existing publication. 2. Select the publication. 3. Click view.	Publication exists, and the user has no permission to access it.	Publication details or documents are successfully displayed to the user.	The publication is successfully displayed to be viewed.	PASS	
	011	Test whether a user can download the publication.	1. Search for an existing publication. 2. Select the publication. 3. Click Download	Publication should exist	Correct publication is downloaded successfully.	The manuscript is successfully downloaded.	PASS	

Requirements Traceability

Req ID	Requirement	Class	Code	Test Case ID	Status
REQ001	The system shall authenticate users using valid credentials (login).	User	RepositoryTokenObtainPairView	001	PASS
REQ002	The system shall authenticate users using valid credentials (login).	User	RepositoryTokenObtainPairView	001, 002	PASS
REQ003	The system shall enforce role-based access control	User, UserProfile	UserProfile.role, permission_classes	003	PASS
REQ004	The system shall allow users to create a publication record	Publication	PublicationViewSet.perform_create, PublicationSerializer	004, 005	PASS
REQ005	The system shall allow researchers to upload research papers.	Publication, Document	PublicationSerializer.validate	006, 007	PASS

REQ006	The system shall allow users to search publications.	Publication, SubjectArea	PublicationViewSet.get_queryset	008, 009	PASS
REQ007	The system shall allow authorized users to download or view an accessible publication document.	Publication, Document	PublicationViewSet.view, PublicationViewSet.download	010, 011	PASS
REQ010	The system shall allow reviewers to view pending publications and approve them	Reviewer, Publication, VerificationRecord	Review_*invitation, review*_decision, refresh_review_status	003, 004, 005	PASS
REQ012	The system shall notify users of submissions, decisions and publications awaiting verification	Notification, UserNotification	notify_user	009, 010, 011	PASS

Increment 2: additional functionality

Increment 2 added six capabilities on top of the core:

- In-app notifications for submissions, approvals, rejections and returns for correction, and notifications to reviewers when a publication needs verification.
- Document validation: accepted file types and content checks.
- Reviewer comments and rejection feedback, so authors can review feedback and resubmit.

Final Increment: enhancements

Enhancement	Status	Notes
Keywords on publications	Delivered	
Email notification	Delivered	Each notification is saved in-app and emailed. If the email fails, the in-app notification is kept and the user is warned.
Student to supervisor mapping	Delivered	
Workflow and history of a publication	Partly delivered	The invitation, decision, revision and resubmission workflow works. A per-version history is not in the models reviewed.
Search parameters as separate fields	Partly delivered	Title, abstract, subject area, author and type are separate filters. A date filter such as the last 5 years is missing.
Summary or overview of publication information	Delivered	Admin overview and PDF extract, described below.
Order of submissions at the supervisor	Delivered	

Reviewer feedback validation	Delivered	Feedback of at least 5 characters and at most 1,000 words is required when requesting revisions.
Abstract word limit	Delivered	Abstracts must be between 150 and 250 words.
Predefined reviewer mapping	Delivered	

Test and traceability

Increment 1 results

Requirement	Test cases	Recorded result
REQ001 Register	TC-001	Pass
REQ002 Login	TC-002, TC-003	Pass
REQ003 Role-based access	TC-004, TC-013, TC-014	Pass
REQ004 Create record	TC-005, TC-006, TC-016	Pass
REQ005 Upload	TC-007, TC-008	Pass
REQ006 Search	TC-009, TC-010	Pass

REQ007 View and download	TC-011, TC-012	Pass
REQ008 Audit trail	TC-015	Not run

Tests for the final increments

ID	Req	Test	Expected Result	Result
TC-021	REQ016	As admin, open the Overview tab	Users by type and publications by type are shown, and the totals match the database	Pass
TC-022	REQ016	As admin, click Extract PDF report	A PDF named unipub-summary with today's date downloads, and its figures match the screen	Pass
TC-023	REQ016	As a student, request the PDF endpoint directly	Access refused with 401 or 403, and no PDF is returned	Pass
TC-024	REQ016	Open the Overview tab with no publications	Counts show 0 and the PDF still generates	Pass

Requirements to design to code to test

Req	Design Element	Code	Tests	Status

REQ001	User.register(), university directory check	UserRegistrationSerializer, UserRegistrationView	TC-001	Pass
REQ002	User.login()	RepositoryTokenObtainPairView	TC-002, TC-003	Pass
REQ003	User.role, permissions	UserProfile.role, permission_classes, PublicationViewSet.get_queryset	TC-004, TC-013, TC-014	Pass
REQ004	Publication, create and edit	PublicationViewSet.perform_create and perform_update, PublicationSerializer	TC-005, TC-006, TC-016	Partial
REQ005	Document.upload()	PublicationSerializer.validate (PDF type and content)	TC-007, TC-008	Pass
REQ006	Search	PublicationViewSet.get_queryset	TC-009, TC-010	Pass
REQ007	Document.download()	PublicationViewSet.download and view	TC-011, TC-012	Pass

REQ009, REQ010	submit_for_verification(), approve()	review_invitation, review_decision, refresh_review_status	TC-011 to TC-15	Pass
REQ011	VerificationRecord comments	review_decision feedback validation	TC-106 to TC-108	Pass
REQ012	Notification	notify_user, UserNotification	TC-109 to TC-111	Pass
REQ015	Advanced search	PublicationViewSet.get_queryset filters	TC-118 to TC-121	Pass
REQ016	Report	AdminAnalyticsView, AdminAnalyticsPdfView, AdminPortal Overview	TC-122 to TC-125, TC-201 to TC-204	Pass

Additional Functionalities Working

Incremental 2

Email linked to System

Notification for researchers' submissions, approvals, rejections and returns to correct. Reviewers' notification for a new publication that needs verification.

Document Validation Security

Restriction of accepted file types and file sizes. Validation of file contents to protect restricted documents from unauthorized access.

Reviewer comments and Rejections Feedback

Reviewers are required to specify a reason for rejection and are expected to leave feedback for researchers. Researchers are expected to review and resubmit their publication.

History

A researcher uploaded a new publication that was revised. Submission, and review history for each version of its publication link.

Advanced Search and Filtering Publication

Authorize users to search by Keywords, title, author, year of publication, subject area, type of publication. Sorting and pagination is recommended essential techniques for large datasets management in web application and REST API.

Upgraded Administrative Reports

Prompt reports of publications showing, subject area, author, publication type, year, and approval status. Include CSV and PDF exports.

INDIVIDUAL CONTRIBUTIONS

1. Lebogang Keiponye (github name: Fenaah)

Implemented:

- Abstract limit or word count to be atleast 150 to 250 words.
- Choice of publications for the students (thesis and dissertation) and lecturers (conference and articles).
- Order of submission, so that the publications that have long been submitted to be at the top and the most recent one's to be at the bottom.
- Displayed the submission date and change of UI, to reduce a lot of white space.
- Validation of reviewer feedback.
- Search by full name, and filter by years.
- How versions are stored.
- Add the filter cards, so that if you press the approved card it, takes you to all the approved and published, publications.
- One author for dissertation and thesis.
- Tested the system and wrote the test cases.

2. Mmoloki Morebodi(git hub handle: Nickie-NicTech,nickie_ui)
   Implemented:
   - Application base ui and backend including:

▪ Django Rest API setup ,database setup
▪ minimal shared UI and backend for search, registration,login and publication creation

- multirole authentication .
- Reviewer Dashboard for role LECTURER
- Repository DashBoard for all users including user profile modelling
- InApp Notifications and API
-

3. Thabo Motlhagodi
   Implemented :
   - Email notifications and Django mailer setup
   - Base Ui for Admin Panel

4. Popagano Batendi
   Implemeted automatic mapping of the reviewers through the simulation of the ASAS interfacing system, including all the validations and exception handling.